registered in listing 33.7. This creates a pipeline similar to that shown in figure 33.6.

Listing 33.10 Registering a custom handler in `Startup.ConfigureServices`

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
builder.Services.AddTransient<ApiKeyMessageHandler>();          ❶

builder.Services.AddHttpClient<ExchangeRatesClient>()
    .AddHttpMessageHandler<ApiKeyMessageHandler>()              ❷
    .AddTransientHttpErrorPolicy(policy =>                      ❸
        policy.WaitAndRetryAsync(new[] {
            TimeSpan.FromMilliseconds(200),
            TimeSpan.FromMilliseconds(500),
            TimeSpan.FromSeconds(1)
        })
    );
```

❶ Registers the custom handler with the DI container

❷ Configures the typed client to use the custom handler

❸ Adds the transient error handler. The order in which the handlers are registered dictates their order in the pipeline.

Whenever you make a request using the typed client `ExchangeRatesClient`, you can be sure that the API key will be added and that transient errors will be handled automatically for you.

That brings us to the end of this chapter on `IHttpClientFactory`. Given the difficulties in using `HttpClient` correctly that I showed in section 33.1, you should always favor `IHttpClientFactory` where possible. As a bonus, `IHttpClientFactory` allows you to easily centralize your API configuration using named clients and to encapsulate your API interactions using typed clients.